ENGLISH PRONUNCIATION IMPROVEMENT ASSISTANT

Project Report

MLLP EXPERIENTIAL LEARNING

Submitted by: Niharika Sahay
Department: Electronics and Computer Science Engineering
Institution:KIIT University
Academic Year: 2026–2027

Project Description

The English Pronunciation Improvement Assistant is a Python-based desktop application designed to help English learners practise speaking through speech recognition, text comparison, and text-to-speech generation.

The application allows users to speak a given English word through a microphone. The recorded speech is processed using OpenAI Whisper to generate a text transcription. The recognized text is then compared with the reference text using the RapidFuzz library to calculate a similarity score. The application also generates reference audio using a text-to-speech engine, allowing learners to listen to an example of the expected pronunciation.

2. Problem Statement and Objectives

2.1 Problem Statement

English is one of the most widely used languages for communication in academic and professional environments. However, many learners face difficulties while speaking English, particularly when pronouncing unfamiliar words and sentences.

Traditional learning methods may not always provide immediate feedback on a learner's spoken responses. Students may find it difficult to identify whether their spoken words match the expected sentence.

There is a need for an interactive system that allows learners to practise speaking English, recognizes their speech, compares their responses with reference text, and provides an example of the expected pronunciation.

The proposed English Pronunciation Improvement Assistant addresses this problem through an integrated speech-processing application.

2.2 Motivation

The motivation behind this project is to make English speaking practice more accessible and interactive.

The application provides an environment in which students can practise independently without requiring a teacher to evaluate every attempt.

By combining speech recognition and text-to-speech technologies, the system aims to support repeated listening and speaking exercises.

2.3 Objectives

The main objectives are:

1. To develop an interactive application for English speaking practice.
2. To capture spoken English using a microphone.
3. To convert recorded speech into text using an automatic speech recognition model.
4. To compare recognized text with a predefined reference sentence.
5. To calculate a text similarity score.
6. To generate reference speech using a text-to-speech system.
7. To provide a simple interface for practising English words and sentences.

2.4 Expected Outcomes

The expected outcome is a functional desktop application that:

- Records the user's spoken English.
- Produces a text transcription of the recorded speech.
- Compares the transcription with the reference text.
- Displays a similarity score.
- Allows the user to listen to synthesized reference audio.

The system is intended to serve as a basic English speaking practice assistant and a foundation for more advanced pronunciation assessment systems.

3. Proposed Methodology and Processing Pipeline

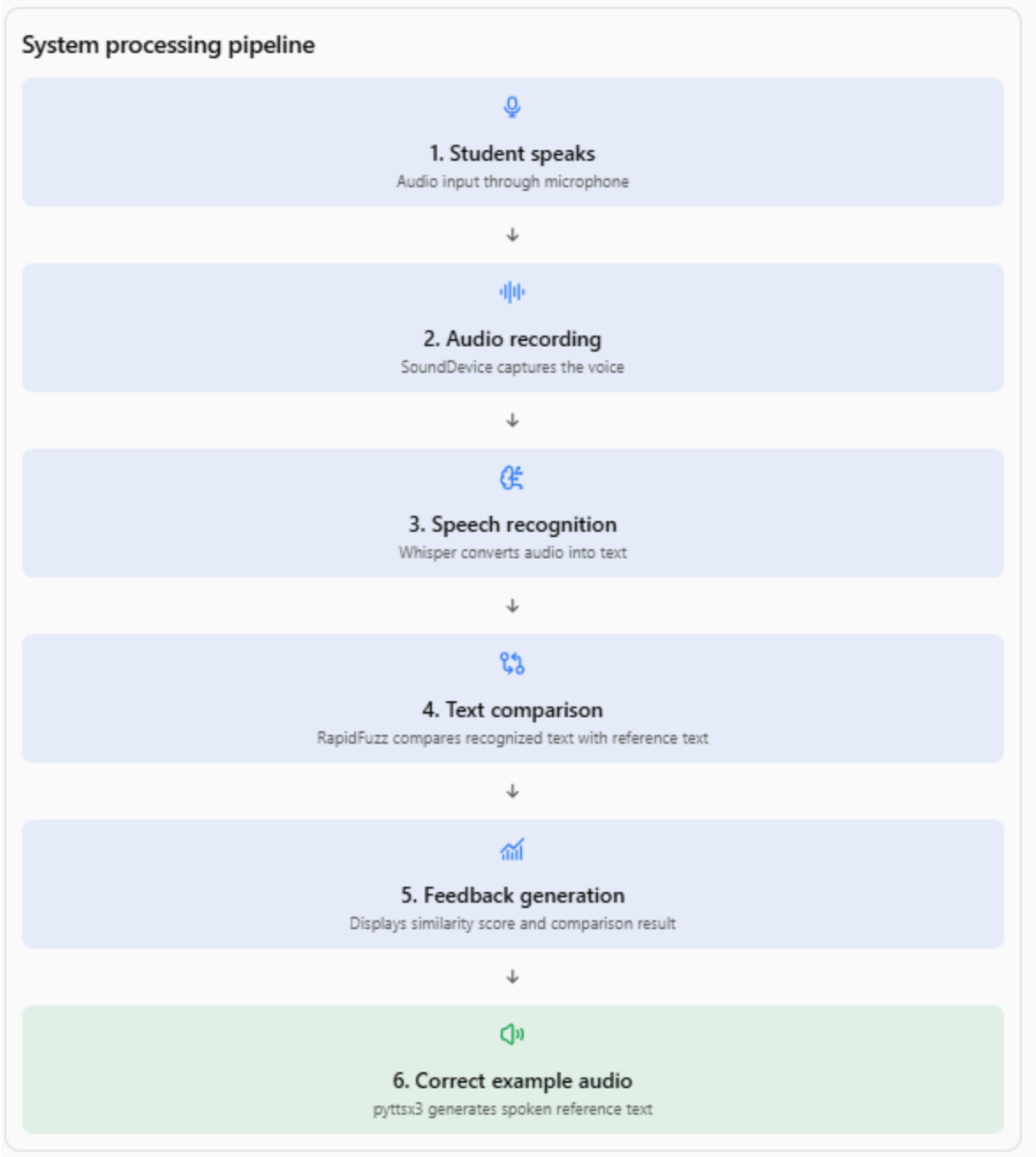
3.1 Overview

The proposed system follows a sequential audio-processing and text-comparison pipeline.

The user provides spoken English through a microphone. The recorded audio is passed to a speech recognition model, which converts it into text. The recognized text is compared with the reference text to calculate a similarity score. Finally, the reference text is converted into speech so that the user can listen to an example.

3.2 Processing Pipeline Block Diagram

Figure 1: System Architecture and Processing Pipeline



3.3 Step-by-Step Explanation

Stage 1: Input

The application consists of different words segregated according to levels in spoken English. It displays the word in random order to act as a reference word.

Stage 2: Audio Recording

The user speaks the reference word through a microphone. The SoundDevice library captures the audio and makes the recording available for processing.

Stage 3: Audio Processing

The recorded audio is prepared in a format suitable for speech recognition. FFmpeg is used by the Whisper audio-processing workflow to handle supported audio formats.

Stage 4: Speech Recognition

OpenAI Whisper processes the recorded audio and generates a textual transcription.

For example, if the user speaks: ‘Weather’

Whisper attempts to convert the spoken sentence into the corresponding text.

Stage 5: Text Comparison

The recognized transcription is compared with the reference word using RapidFuzz.

RapidFuzz provides string-similarity algorithms that help determine how closely two text strings match.

Stage 6: Similarity Score Generation

The application calculates a similarity score based on the difference between the reference text and the recognized transcription.

A higher similarity score indicates a closer textual match.

Stage 7: Feedback and Audio Output

The application displays the comparison result and provides reference audio through pyttsx3.

The learner can listen to the synthesized sentence and repeat the exercise.

3.4 Input, Intermediate Processing and Output

Category	Description
Input	Reference English word and recorded student speech
Intermediate processing	Audio capture, speech recognition and text comparison
Recognition output	Text generated by Whisper

Comparison output	Similarity score generated using RapidFuzz
Final output	Displayed result and synthesized reference audio

4. Models and Tools Used

The application uses a combination of pretrained machine learning models and Python libraries.

4.1 Machine Learning Model

OpenAI Whisper

Whisper is an automatic speech recognition (ASR) model developed by OpenAI.

It is designed to recognize spoken language and convert audio into text.

In this project, Whisper processes the recorded English speech and generates a transcription that is subsequently compared with the reference sentence.

- Model: Whisper
- Model type: Automatic Speech Recognition
- Developer: OpenAI
- Input: Recorded audio
- Output: Recognized text
- Model source: https://github.com/openai/whisper

The repository's GUI source configures the tiny.en model in the version used for this demonstration. This is an English-focused Whisper model designed for relatively lightweight speech recognition.

Hugging Face model reference: https://huggingface.co/openai/whisper-tiny.en

4.2 Text Similarity Algorithm

RapidFuzz

RapidFuzz is a Python library used for fast string matching and fuzzy text comparison.

It is used to compare the recognized transcription against the reference text.

The similarity score helps indicate how closely the recognized words match the expected words.

Official source: https://github.com/rapidfuzz/RapidFuzz

4.3 Libraries and Frameworks

Technology	Purpose
Python	Main programming language
OpenAI Whisper	Speech recognition
RapidFuzz	Text similarity calculation
SoundDevice	Audio recording
NumPy	Numerical and audio data processing
SciPy	Scientific and signal-processing utilities
pyttsx3	Text-to-speech generation
Tkinter	Graphical user interface
FFmpeg	Audio processing support
PyTorch	Deep learning framework used by Whisper

4.4 Implementation Environment

- Programming Language: Python 3.8 or later
- IDE: Visual Studio Code
- Operating System: Windows
- Interface Framework: Tkinter
- Model Execution: Local computer using CPU-compatible PyTorch installation
- Audio Input: Computer microphone

4.5 Pretrained Models and Data Sources

The project uses the pretrained Whisper model rather than training a new speech recognition model from scratch.

The GitHub repository also contains a data directory with application data, including word-list resources. The README describes the word database as CEFR-levelled, which organizes practice vocabulary according to language proficiency levels.

Dataset and application data:

No custom machine learning model training is performed as part of the current prototype.

5. Implementation and Usability

5.1 Implementation Procedure

The application is implemented using Python and is divided into multiple modules.

The main components are:

1. Main Application Module

The main.py file acts as the entry point and launches the application.

2. GUI Module

The gui_tkinter.py file implements the graphical user interface using Tkinter.

It allows users to interact with the application and perform pronunciation practice.

3. Audio Recording Module

The audio_recorder.py file manages microphone-based audio recording using SoundDevice.

4. Speech Recognition and Scoring Module

The scorer.py file contains the speech recognition and scoring logic. It uses Whisper to transcribe recorded speech and RapidFuzz to compare the resulting text with the reference.

5. Text-to-Speech Module

The speaker.py file provides speech synthesis functionality through pyttsx3.

5.2 Installation Requirements

The following software and libraries are required:

- Python 3.8 or later
- FFmpeg
- PyTorch
- OpenAI Whisper
- SoundDevice

- NumPy
- SciPy
- RapidFuzz
- pyttsx3

5.3 Instructions for Running the Application

Step 1: Download the repository

Open the GitHub repository: https://github.com/Niharika-Sahay/mllp-project.git

Download the ZIP file and extract it, or clone it using:

Step 2: Open the project

Open the extracted project folder in Visual Studio Code.

Step 3: Install dependencies

Open the VS Code terminal and execute: pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu

Then execute:

pip install "numpy<2"
pip install -r requirements.txt

Step 4: Install FFmpeg

Download FFmpeg from: https://ffmpeg.org/download.html

Install it and add its bin directory to the Windows PATH environment variable.

Step 5: Launch the application

Run: python main.py

Step 6: Practise pronunciation

Use the application's interface to select a reference word or sentence, record speech, view the recognized text and similarity score, and listen to the reference audio.

5.4 Practical Applications and Usability

The application can be used in several educational situations:

1. **Self-learning:** Students can practise English independently.
2. **Language Learning:** Beginners can practise common English words and sentences.
3. **Classroom Demonstrations:** Teachers can demonstrate basic speech recognition and text comparison.
4. **Speaking Practice:** Learners can repeat sentences and compare their recognized responses.
5. **Pronunciation Awareness:** Students can become more familiar with the difference between expected and recognized text.

The desktop-based interface makes the application accessible to users with basic computer knowledge.

6. Results and Sample Outputs

6.1 Sample Input and Output

The application was tested using short English words.

The following examples illustrate the system's expected workflow and observed recognition behaviour.

Test Case 1:

Reference Input:

“Whirlwind”

Observed Recognition:

“Whirlwind”

Observed Score:

10/10

Observation:

The speech recognition model correctly identified the spoken word as “Whirlwind.” Since the recognized text matched the reference input, the text similarity algorithm awarded a full score of 10/10. This indicates successful recognition in this test case.

Test Case 2:

Reference Input:

"Sailor"

Observed Recognition:

"See ya lord"

Observed Score:

4/10

Observation:

The speech recognition model incorrectly transcribed the spoken word. This affected the similarity score because the comparison algorithm evaluates the recognized text rather than the actual phonetic pronunciation.

Overall observation: The two test cases show that the system's score depends heavily on speech recognition accuracy. When Whisper recognizes the word correctly, the similarity score is high; when it misrecognizes the word, the score can be low even if the pronunciation was understandable.

6.2 Performance Evaluation

The prototype was evaluated qualitatively through microphone-based speaking tests.

The following observations were made:

- The application successfully captures spoken audio.
- Whisper generates text transcriptions from recorded speech.
- RapidFuzz provides a similarity score based on the recognized and reference text.
- The TTS module generates example audio from reference text.
- Recognition quality varies depending on the spoken word, microphone quality, speaking speed and background noise.

No formal benchmark dataset or large-scale accuracy evaluation was conducted.

6.3 Screenshots and Demonstration Evidence

Figure 2: Application Interface

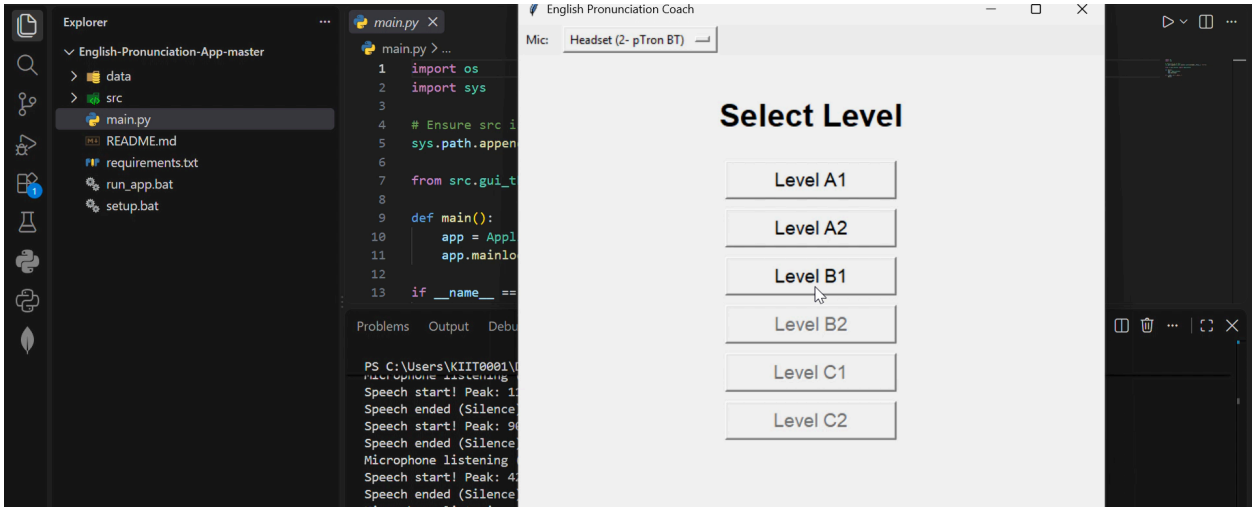
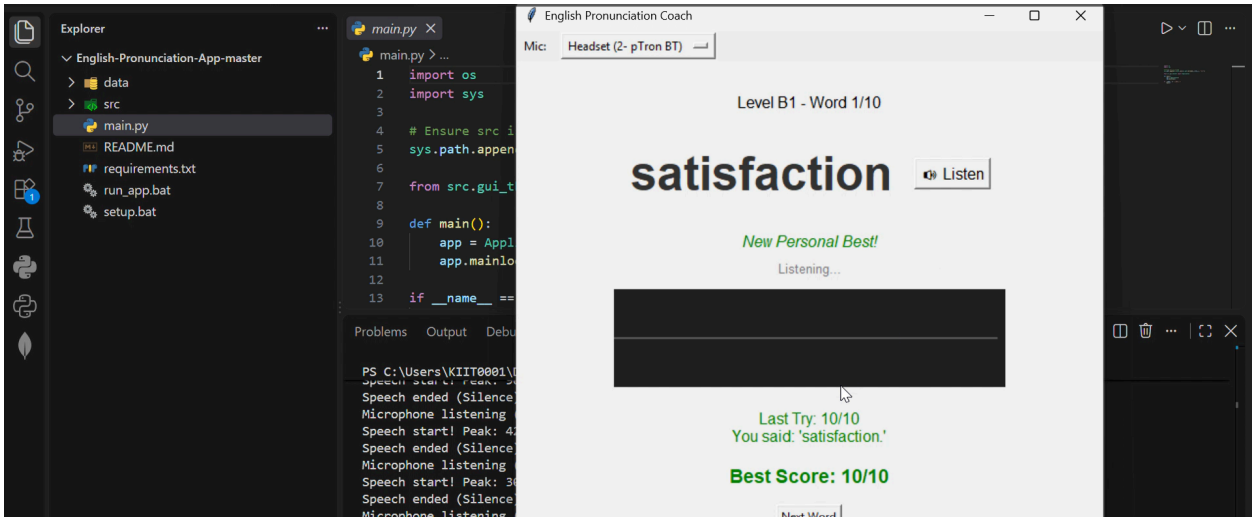


Figure 3: Audio Recording and Recognition



6.4 Limitations

The current implementation has several limitations:

1. **Speech Recognition Errors:** Whisper may incorrectly recognize words, especially when speech is unclear or the recording quality is poor.
2. **Text-Based Evaluation:** RapidFuzz compares text strings and does not directly evaluate phoneme-level pronunciation accuracy.
3. **Accent Variations:** Different English accents may affect the recognition output.
4. **Background Noise:** Environmental noise and poor microphone quality may reduce transcription reliability.

5. **Limited Feedback:** The application does not provide detailed information about incorrect tongue placement, individual phonemes, word stress or intonation.
6. **Basic Speech Synthesis:** The generated reference audio is synthesized speech and may not represent every natural English accent.

7. Conclusion and Future Scope

7.1 Conclusion

The English Pronunciation Improvement Assistant demonstrates how speech recognition, text similarity and speech synthesis technologies can be combined to create an interactive English speaking practice application.

The system allows users to record spoken English, convert speech into text using OpenAI Whisper, compare recognized text with reference sentences using RapidFuzz, and listen to synthesized reference audio using pyttsx3.

The project successfully demonstrates the integration of multiple Python libraries into a desktop-based language-learning prototype.

Although the current system has limitations in evaluating actual pronunciation accuracy, it provides a foundation for developing more advanced pronunciation learning applications.

7.2 Future Scope

The application can be improved in several ways:

1. **Phoneme-Level Assessment:** Integrating phonetic analysis to identify incorrectly pronounced individual sounds.
2. **Word-Level Feedback:** Highlighting specific words that differ from the reference.
3. **Stress and Intonation Analysis:** Evaluating word stress, sentence rhythm and intonation.
4. **Improved Speech Recognition:** Using larger and more accurate speech recognition models.
5. **Accent Support:** Supporting different varieties of English pronunciation.
6. **Personalized Learning:** Maintaining practice history and tracking improvements over time.
7. **Advanced Pronunciation Scoring:** Integrating dedicated pronunciation assessment models rather than relying only on text similarity.

These improvements could transform the current prototype into a more comprehensive English pronunciation learning platform.

8. References and GitHub Repository Link

8.1 GitHub Repository

Link: https://github.com/Niharika-Sahay/mllp-project

8.2 References

1. OpenAI. *Whisper: General-purpose Speech Recognition Model.* GitHub Repository. https://github.com/openai/whisper
2. OpenAI. *Whisper Tiny English Model.* Hugging Face. https://huggingface.co/openai/whisper-tiny.en
3. RapidFuzz. *RapidFuzz: Fuzzy String Matching in Python.* https://github.com/rapidfuzz/RapidFuzz
4. Python Software Foundation. *Python Documentation.* https://docs.python.org/3/
5. SoundDevice. *Python Audio Recording and Playback Documentation.* https://python-sounddevice.readthedocs.io/
6. pyttsx3. *Text-to-Speech Conversion Library.* https://pyttsx3.readthedocs.io/
7. FFmpeg. *Official Documentation and Downloads.* https://ffmpeg.org/documentation.html
8. Python Tkinter. *Graphical User Interface Documentation.* https://docs.python.org/3/library/tkinter.html
9. NumPy. *Numerical Computing Library.* https://numpy.org/doc/
10. SciPy. *Scientific Computing Library.* https://docs.scipy.org/doc/scipy/